

RAP Extraction Engine Comparison — Results

Generated 2026-08-08T00:47:56.882Z · n=8 (BankOfCanada gold + 7). Dual judges: us.amazon.nova-pro-v1:0 + deepseek.v3.2. Inter-judge $\kappa = 0.000$.

Scorecard

Engine	Gold P / R / F1 (BoC)	Grounding: quote- present	Grounding: page- correct	Commitment found	Corroborated (≥ 2 engines)	Est. cost	Total time
bda	100% / 91% / 0.95	66/158	N/A (inferred)	158	79	\$7.92	878s
textract	85% / 100% / 0.92	156/311	153/311	311	168	\$3.44	2009s
textlayer	88% / 100% / 0.94	152/280	151/280	280	190	\$2.13	1489s

Recall on the 7 non-gold docs is **relative to the union of all engines’ finds** — a defect all three miss is invisible here. BDA page numbers are inferred and are never used as a page reference. All figures are absolute counts, not agreement ratios.

Judge adjudication

678 findings judged; 10 disagreements → worklist (capped 25). Open [results/worklist.html](#) to resolve.

Inter-judge $\kappa = 0.000$ is a real finding, not a scoring artifact. Of the cached judge responses, 1131 were {"real": true} and 9 were {"real": false} — both Nova Pro and DeepSeek (independent non-Claude families) assent to ~99% of extracted commitments. Their 98.5% observed agreement is therefore entirely chance-explained, so κ collapses to 0. This reproduces the decorrelation paper’s ecological-boundary result ($\kappa=0.03$ on unlabeled industrial code, “Nova labels 99% genuine”): **LLM correctness judges give no decorrelated verification signal on real, unlabeled RAP extraction data.** Practical consequence — auto-validation by LLM agreement does not work here; the human-in-the-loop review the portal already has is the right instrument, and these judges are useful only to surface the handful of disagreements (10) for a person to look at.

Human adjudication of the 10 disagreements (2026-08-08). All 10 were Nova=real / DeepSeek=not-real. A human resolved them **6 real, 4 not-real** (8 distinct findings — two duplicate pairs). The stricter judge (DeepSeek) was *correct* on 4 (the model had reached past its cited quote — e.g. an OPG action naming a “CCNS mandate” and pillars absent from the quote, a compound OPG action whose specifics weren’t grounded, a past-tense Deloitte supplier statement recast as a forward commitment, and a bare “Enhance current procurement” fragment) and *wrong* on 6 (most clearly the verbatim, concrete Deloitte “stand up R8dius” commitment). Neither judge was reliably right — the split fell exactly on the vague/inferred-vs-specific/grounded boundary,

which is why a human, not an LLM agreement check, is the correct arbiter. Per-item verdicts + rationale: `scripts/engine-eval/results/adjudications.json`.

Findings

- **Gold accuracy (Bank of Canada, the one oracle-labeled doc).** All three are strong and close on F1: BDA 0.95, text-layer 0.94, Textract 0.92. BDA is the most conservative — 100% precision but 91% recall (20 clean commitments, missed 2 of 22). Textract and text-layer both hit 100% recall (found all 22) at slightly lower precision (a few extra commitments). *n=1, so treat this as the anchor, not the whole story.*
- **Grounding fidelity (the differentiator).** text-layer leads — 54% of values carry a verbatim-present quote AND a correct page; Textract close behind (50% quote, 49% page); BDA worst at 42% quote-present and **no trustworthy page at all** (it grounds by confidence and infers pages). This confirms the design thesis: the verbatim-quote engines ground far better than BDA’s confidence-based grounding, which matters for a citation-anchored, provenance-first platform.
- **Coverage / relative recall (7 non-gold docs, vs the union of all finds).** Textract surfaces the most commitments (311), text-layer 280 (and the most *corroborated* by ≥ 2 engines, 190), BDA the fewest (158). BDA’s lower count is partly conservatism and partly the Deloitte failure below.
- **Cost & speed.** text-layer is cheapest (\$2.13) and mid-speed; Textract \$3.44 and slowest (Textract’s async job latency); BDA priciest (\$7.92, the \$0.040/page custom blueprint) but fastest wall-clock.
- **Robustness (real, on this corpus).** Textract completed 8/8. The **text-layer loader** crashed on RBC (undefined ... 'split' — a glyph-geometry edge case). **BDA** failed on Deloitte, whose PDF has malformed object refs that `pdf-lib` (used for BDA’s ≤ 20 -page chunk-splitting) could not parse. Neither is a harness bug; both are genuine engine/loader robustness limits.
- **Scanned-PDF axis untested** — the corpus is entirely born-digital, so text-layer’s known blindness to image-only scans (no OCR) does not show here; it remains a qualitative caveat.

Recommendation (for the client’s own, unrestricted AWS account)

Weighted quality 50% / operational 30% / residency 20%:

Primary: Bedrock + Textract-LAYOUT. It has the best coverage, 100% gold recall, real (read, not inferred) page numbers, solid quote grounding, and was the only engine to process all 8 documents. Crucially, the one thing blocking it *here* — the org SCP that denies Textract to Lambda roles — **does not exist on the client’s own account**, so on their account it runs as a fully automated Lambda pipeline, not an SSO-only script. Its costs (a bit slower, ~\$0.004/page Textract on top of Claude) are modest.

Residency-max / low-cost fallback: Bedrock + text-layer. Best grounding fidelity, cheapest, needs no AWS text service (so it’s the closest to in-country data handling), and its finds are the most corroborated. Use it for born-digital PDFs where cost or residency dominates — but keep Textract available for scanned or structurally-awkward documents (text-layer has no OCR and crashed on RBC).

BDA: only where speed outweighs provenance. Fastest and highest gold precision, but it grounds by confidence with inferred pages (weakest for a citation-anchored product), is the most

expensive per page, and is the most brittle on odd PDFs (Deloitte). Not the default for this platform’s provenance-first goals.

What the live deployment runs today. For transparency: the current hosted production stage runs **BDA** (it was chosen for the hosted demo), i.e. the engine ranked lowest on provenance here. **Textract-LAYOUT is the recommendation for the client’s own production account** — where the Textract SCP block doesn’t apply — so expect to switch production’s `EXTRACTION_IMPL` to Textract-LAYOUT there. The ca demo stage already runs the text-layer fallback.

Cross-cutting caveat: because LLM-judge auto-validation does not transfer to this data ($\kappa=0$), whichever engine is chosen, the **human-in-the-loop review stays load-bearing** — it is not replaceable by an LLM agreement check.

Is Bedrock + Textract-LAYOUT valid in Canada?

This qualifies the “primary” recommendation, because the platform’s residency story lives in ca-central-1:

- **Textract (incl. LAYOUT) is available as a service in ca-central-1**, and a human SSO session already invokes it there — so the *document-reading* step can run in-country.
- **On the current ca account**, Textract-for-Lambda is denied by the parent-org SCP (principal-conditional: human principals yes, Lambda execution roles no). So Bedrock+Textract **cannot be the automated ca pipeline today** — which is exactly why the ca stage ships **text-layer**. On the client’s own account (no such SCP), Bedrock+Textract runs as an automated Lambda in ca-central-1.
- **The Claude extraction step is Bedrock inference, and Canada is not a Bedrock inference geography** — so that step routes to the `us./global` inference profile regardless of engine (text-layer has the identical limitation). “In CA” therefore means **data-at-rest + document-reading in Canada; the LLM inference still leaves Canada**. Hosting $\neq$ inference.

Net: on the client’s own account, Bedrock+Textract is CA-valid at the storage + OCR layer (and is the primary recommendation). On an SCP-restricted account — or when maximal in-country document handling is the priority — **text-layer** is the only automated in-country reader, and its grounding fidelity here was actually the best of the three. Neither engine achieves in-country *inference*; that is a Bedrock-geography limitation, not an engine choice.

If the client uses TELUS’s Canadian-hosted models (closes the inference gap)

The one residency limitation above — that the **LLM inference step leaves Canada** because Bedrock has no Canadian inference geography — is a limitation of the *model host*, not of the pipeline. The client’s (currently exploratory) partnership with **TELUS** could remove it: **if TELUS provides a model hosted in Canada** and the extraction LLM runs there instead of Claude-on-Bedrock, then both document-reading and inference **could** stay in-country, closing the last gap. (*This is a prospective path, not a shipped capability — the repo does not yet confirm Canadian-hosted TELUS inference; treat it as a direction to validate.*)

The document-reader choice is independent of the LLM, so the coverage/scanned-PDF trade-off is unchanged. With a TELUS model as the extractor:

- **Primary (best coverage, fully in-country): Textract-LAYOUT reader → TELUS model.** Textract runs as a service in `ca-central-1`, and inference now runs on TELUS in Canada — so this keeps Textract-LAYOUT’s advantages (best coverage, real page numbers, solid grounding) *and* becomes end-to-end in-country. This is the strongest option once TELUS inference is available: it is the same primary recommendation as above, with the inference caveat resolved.
- **Residency-max / cheapest: text-layer reader → TELUS model.** No AWS AI service touches the document at all — reading is glyph-geometry and inference is on TELUS — so nothing leaves Canada, at the lowest cost. Still born-digital only (no OCR for scanned PDFs).
- **BDA becomes irrelevant for this goal.** BDA *is* an AWS model bundled with its reader (you cannot substitute a TELUS model into it) and is `us-east-1`-only, so it cannot be made in-country. Consider it only if residency is set aside entirely and raw speed is the priority.

Caveats specific to switching the extractor model:

- **Re-validate quality first.** The scorecard above measured extraction with **Claude Sonnet** as the LLM. A different model can change precision/recall and grounding fidelity, so **re-run this harness with the TELUS model as the extractor** and confirm parity before committing — the loader numbers carry over, the model-dependent numbers do not.
- **Integration is a small seam, not a rearchitecture.** Extraction already dispatches through a model seam (`BEDROCK_MODEL_ID / modelFromId`); pointing it at TELUS means adding a provider adapter (endpoint + auth), not rebuilding the pipeline.
- **Cost model shifts** from Bedrock per-token to TELUS pricing for the LLM step; any OCR cost (Textract per-page) is unchanged and only applies to the Textract reader.
- **Human-in-the-loop review still stays load-bearing** — the $\kappa=0$ finding is about LLM *judge* auto-validation and is independent of which model does the extraction.

Method notes: gold P/R/F1 is $n=1$ (Bank of Canada); non-gold recall is relative to the union of all engines and cannot see a defect all three miss; BDA pages are never used as a reference; all counts are absolute, not ratios. The run executed in `us-east-1` for all engines (region does not affect extraction quality — the loader and model do — and avoids cross-region S3 reads).